AIML PROJECT REPORT

Smart Study Assistant

Mood & Focus Tracker

Submitted by

[KISLAY KUMAR]
Roll No: [25BAI10313]
[CSE BTECH] • [VIT BHOPAL]

Academic Year 2025–26

1. Abstract

This project presents a Smart Study Assistant — a lightweight, rule-based AI application designed to help students make more informed decisions about their study habits. The system takes two simple inputs — mood and focus level — along with session duration and subject, and produces a quality rating, personalised advice, and a visual trend of performance over time.

The core AI logic is a weighted scoring classifier with a deterministic advice tree, chosen deliberately over machine learning methods because the individual-user dataset is far too small for a model to generalise meaningfully. The project is delivered in two interfaces: a terminal application written in Python and a browser-based dashboard in HTML, CSS, and JavaScript.

The assistant also integrates a Pomodoro timer, subject-specific study tips drawn from evidence-informed strategies, and a trend analyser that compares recent sessions to tell the user whether their performance is improving or declining.

2. Introduction

Students frequently sit down to study without pausing to assess whether their mental state is ready for focused work. Fatigue and distraction are the two most consistent enemies of effective studying, yet most study tools — timers, planners, flashcard apps — do nothing to account for them. They treat every session as equivalent, which they are not.

This project attempts a different angle. Instead of helping students study harder, it helps them study smarter by tracking the quality of their sessions over time and nudging them toward conditions that have worked for them before.

2.1 Motivation

The idea came from a practical frustration: spending two hours on a subject and retaining almost nothing because the studying happened while exhausted. A tool that flags this pattern and suggests a break or a different time slot could meaningfully change how a student plans their week.

2.2 Objectives

- Build a system that rates study session quality from mood and focus inputs
- Generate practical, condition-specific advice — not generic productivity platitudes
- Track sessions over time and surface trends the user might not notice themselves
- Provide a Pomodoro timer integrated with the session logging workflow
- Keep the application dependency-free and runnable on any student's machine

3. System Design

3.1 Architecture Overview

The application has three layers: input collection, the AI classifier, and output rendering. There is no server, no database, and no internet requirement. All data is stored locally — in a JSON file for the terminal app, and in localStorage for the web dashboard.

Layer	Terminal App	Web Dashboard	Technology
Input	CLI prompts	HTML form + sliders	Python / JS
AI Logic	app.py classifier	JavaScript classifier	Rule-based
Storage	study_log.json	localStorage	JSON
Output	ASCII chart + text	Canvas chart + table	Terminal / HTML

3.2 The Classifier

The session quality classifier works as follows:

1. The user provides mood (1–5), focus (1–5), duration (minutes), and subject.
2. A weighted score is computed: score = (mood × 0.4) + (focus × 0.6)
3. A duration penalty is applied for sessions exceeding 90 minutes, reflecting cognitive fatigue.
4. The score maps to one of four quality tiers: Excellent, Decent, Struggling, or Burnout Risk.
5. An advice tree evaluates each condition independently and appends a relevant suggestion.

The 0.4/0.6 weighting between mood and focus reflects a deliberate design choice: focus is more directly correlated with learning effectiveness than mood. A student can feel tired but still concentrate; the reverse is less productive.

3.3 Trend Analysis

After three or more sessions are recorded, the trend analyser compares the oldest and newest scores in the last three sessions. A delta greater than +0.5 reports an upward trend; below -0.5, a downward trend; otherwise, stable. This is deliberately simple — with small datasets, complex statistical methods would produce misleading confidence.

3.4 Subject Tips

Each of the six supported subjects (math, science, english, history, coding, general) has three tips drawn from established study strategy research. One is selected randomly at

the end of each session. These are not AI-generated — they were written deliberately to be specific and actionable rather than generic.

4. Implementation

4.1 Terminal Application (app.py)

The terminal app is a single-file Python script with no external dependencies. It uses only the standard library (json, os, time, datetime, random, math, collections). Key functions include:

- classify_session() — the main classifier returning quality, score, icon, and advice list
- naive_trend() — compares last 3 session scores and returns a trend string
- ascii_bar() — renders a text-based bar chart without matplotlib
- run_pomodoro() — a blocking timer with a live progress bar using carriage returns
- weekly_summary() — filters the last 7 days and computes aggregated statistics
- print_history_chart() — displays the last 7 sessions as a formatted ASCII table

Data is stored as a JSON array in study_log.json. Each entry is a dictionary with timestamp, subject, mood, focus, duration_mins, quality, score, and notes fields.

4.2 Web Dashboard (dashboard.html)

The dashboard is a self-contained HTML file. No frameworks, no bundler, no server. It uses:

- Vanilla JavaScript for all application logic, mirroring the Python classifier exactly
- HTML5 Canvas API for the focus trend chart — hand-drawn using Bezier curves and a gradient fill
- CSS custom properties and a carefully chosen dark colour palette (navy + indigo accent)
- Google Fonts (Syne + DM Mono) loaded via @import for typographic character
- Local Storage for persistence across browser sessions

4.3 Design Choices

Two deliberate choices are worth explaining. First, the decision to build a rule-based classifier rather than use scikit-learn or a neural network. With one person logging maybe 20–50 sessions over a semester, any ML model would have so few training examples that its predictions would be essentially random. The rule-based approach is honest about what it knows and why.

Second, the choice to avoid external libraries entirely. This was partly practical — students submitting project work often run into environment issues — and partly philosophical. A system you can read and verify is more trustworthy than one whose internals are opaque.

5. Results & Sample Output

5.1 Terminal Session

When a user logs a session with mood=4 and focus=5 over 60 minutes of coding:

```
score = (4 × 0.4) + (5 × 0.6) = 1.6 + 3.0 = 4.6
```

The system outputs: quality = EXCELLENT, icon = □, and the advice: "You're in the zone — keep this session going, avoid interruptions!"

5.2 Trend Analysis

Given three consecutive session scores of [2.8, 3.4, 4.1], the trend analyser computes delta = 4.1 - 2.8 = 1.3 > 0.5 and returns: "□ Trending upward — you're improving!"

5.3 Weekly Summary (sample)

Metric	Value	Note
Sessions this week	6	—
Total study time	280 min	~4.7 hrs
Avg mood	3.7 / 5	"good"
Avg focus	3.5 / 5	"focused"
Top subject	Coding	140 min

6. AI / ML Concepts Applied

This project touches on several concepts studied in the AIML course:

6.1 Knowledge Representation

The advice tree and subject tips represent domain knowledge in an explicit, structured form. This is classic expert system design — a branch of AI that predates machine learning and remains highly applicable in narrow domains with well-understood rules.

6.2 Decision Trees (simplified)

The classifier's advice generation is essentially a hand-coded decision tree: each condition (mood ≤ 2, focus ≤ 2, duration > 90, etc.) is a node, and each branch produces a specific output. The difference from a learned decision tree is that the thresholds and branches were set by reasoning about the domain rather than by training on data.

6.3 Weighted Scoring / Feature Weighting

The score formula — (mood × 0.4) + (focus × 0.6) — is an instance of feature weighting, a technique used in both classical AI and ML to encode prior knowledge about the relative importance of different inputs. In a supervised learning setting, these weights would be learned from data. Here they are set by design.

6.4 Time Series Trend Detection

The trend analysis compares ordered time-stamped observations and computes a directional signal. This is a simplified version of the kind of analysis done in time-series forecasting models. The approach chosen (delta between first and last of a window) is analogous to a simple moving average difference.

6.5 Natural Language Output

The system generates natural language advice strings conditionally. While this is not NLP in the generative sense, it does involve selecting and composing text based on structured input — a form of template-based natural language generation that is conceptually related to rule-based chatbot design.

7. Limitations & Future Work

7.1 Current Limitations

- Self-reported mood and focus are subjective — the system cannot verify them
- The advice tree has a fixed set of conditions and will not adapt to unusual patterns
- Trend detection requires at least 3 sessions and only looks at a window of 3
- No cross-device sync — terminal and web data are stored separately
- The Pomodoro timer blocks the terminal; only one thing can happen at a time

7.2 Possible Extensions

- Add sleep hours as an input variable; it is arguably more predictive than mood
- Once 30+ sessions are logged, train a simple linear regression model to predict optimal study conditions
- Add a CSV export for analysis in Excel or pandas
- Multi-user support with separate log files per user

- Browser notification API integration for Pomodoro alerts
- A calendar heatmap view to visualise study consistency across weeks

8. Conclusion

The Smart Study Assistant demonstrates that a useful, practically helpful AI system does not need to be a large neural network or a complex machine learning pipeline. A well-designed rule-based classifier with transparent logic, grounded in a clear understanding of the problem domain, can provide genuine value to its users.

The project achieves all of its stated objectives: it logs sessions, rates them, gives actionable advice, tracks trends over time, and provides a Pomodoro timer — all without any external dependencies, in two complementary interfaces. The code is readable, the logic is explainable, and the system is honest about what it does and does not know.

Perhaps more importantly, it is the kind of tool that a student would actually use, which is ultimately what any AI application should aim to be.

9. References

6. Cirillo, F. (2006). The Pomodoro Technique. FC Garage.
7. Dunlosky, J. et al. (2013). Improving Students' Learning With Effective Learning Techniques. Psychological Science in the Public Interest, 14(1), 4–58.
8. Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
9. Python Software Foundation. (2024). Python 3 Standard Library Documentation. https://docs.python.org/3/library/
10. MDN Web Docs. (2024). Canvas API. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
11. Sweigart, A. (2019). Automate the Boring Stuff with Python (2nd ed.). No Starch Press.